

PROJECT MANAGEMENT SYSTEM

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Aviraj Singh-EN23CS301234

Avishka Vyas-EN23CS301236

Under the Guidance of

Prof. Priyanka Jain

Prof. Kamini Vishwakarma



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

PROJECT MANAGEMENT SYSTEM

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Aviraj Singh-EN23CS301234

Avishka Vyas-EN23CS301236

Under the Guidance of

Prof. Priyanka Jain

Prof. Kamini Vishwakarma



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

REPORT APPROVAL

The project work “Project Management System” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

I/We hereby declare that the project entitled “**Project Management System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘Computer Science and Engineering’ completed under the supervision of Prof. Priyanka Jain and Prof. Kamini Vishwakarma from department of Computer Science and Engineering, Faculty of Engineering, Medicaps University Indore is an authentic work.,
Further, I/we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Aviraj Singh-

Avishka Vyas-

Certificate

I/We, Prof. Priyanka Jain and Prof. Kamini Vishwakarma certify that the project entitled “**Project Management System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Aviraj Singh and Avishka Vyas** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Kamini Vishwakarma

Computer Science and Engineering

Medicaps University, Indore

Prof. Priyanka Jain

Computer Science and Engineering

Medicaps University, Indore

Prof. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Dr. Ratnesh Litoriya**, Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Prof. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Students may write as per their experience.

Aviraj Singh

Avishka Vyas

B.Tech. III Year

(Section-D)

Department of Computer Science & Engineering

Faculty of Engineering

Medicaps University, Indore

Abstract

The Project Management System is a comprehensive web-based application developed to simplify and enhance the process of project planning, task management, and team collaboration. In modern organizations and academic environments, managing multiple projects and coordinating among team members can become complex and time-consuming when performed manually. This system addresses these challenges by providing a centralized digital platform that enables users to efficiently manage all project-related activities in one place.

The primary objective of this system is to improve productivity, ensure transparency, and facilitate effective communication among team members. The application allows users to create, update, and manage projects, assign tasks to individuals, track task progress, and monitor deadlines. It supports multiple user roles such as Admin, Project Manager, and Team Member, each having specific permissions and responsibilities. The Admin manages the overall system and users, the Project Manager handles project creation and task assignment, while Team Members focus on completing assigned tasks and updating their status.

Key features of the system include secure user authentication, role-based access control, task and project management, team coordination, file and document sharing, real-time notifications, and performance tracking through dashboards and reports. The system also incorporates communication features such as comments and feedback mechanisms, enabling better collaboration among users. Notifications and alerts help users stay updated about task assignments, deadlines, and project progress.

From a technical perspective, the system follows a three-tier architecture consisting of the Presentation Layer (frontend), Application Layer (backend), and Data Layer (database). The frontend is developed using technologies such as HTML, CSS, and JavaScript, while the backend can be implemented using Node.js or similar frameworks. The database system, such as MySQL or MongoDB, ensures efficient data storage, retrieval, and integrity. The system is designed to be scalable, user-friendly, and accessible across multiple devices including desktops, laptops, tablets, and smartphones.

Security and performance are key considerations in the system design. The application ensures secure data transmission using HTTPS protocols, implements authentication mechanisms, and protects against common vulnerabilities. Additionally, it is optimized to handle multiple users simultaneously with minimal response time.

In conclusion, the Project Management System provides an efficient and reliable solution for managing projects and tasks. It enhances team collaboration, reduces manual effort, and ensures timely completion of projects, making it highly beneficial for both academic and professional use.

Keywords: Project Management System, Web Application, Task Management, Team Collaboration, Role-Based Access Control, Dashboard, Project Tracking.

Table of Contents

Chapter	Content	Page No.
	Title Page	1
	Report Approval	2
	Declaration	
	Certificate	4
	Acknowledgement	5
	Abstract	6
	Table of Contents	7-8
	List of figures	9
	List of tables	10
	Abbreviations	11
	Notations & Symbols	12
Chapter 1	Introduction (Whichever is applicable)	13-16
	1.1 Introduction	13
	1.2 Literature Review	13
	1.3 Objectives	14
	1.4 Significance	15
	1.5 Research Design	15
	1.6 Source of Data	16
	1.7 Chapter Scheme	16
Chapter 2	REQUIREMENTS SPECIFICATION	17-20
	2.1 User Characteristics	17
	2.2 Functional Requirements	17
	2.3 Dependencies	19
	2.4 Performance Requirements	20
	2.5 Hardware Requirements	20
	2.6 Constraints & Assumptions	20
Chapter 3	DESIGN (Whichever is applicable)	21-30
	3.1 Algorithm (if Applicable)	21
	3.2 Function Oriented Design for procedural approach	21
	3.3 System Design (Whichever is applicable)	22
	3.3.1 Data Flow Diagrams (Level 0, Level1)	22
	3.3.2 Activity Diagram	24
	3.3.3 Flow Chart	25
	3.3.4 Class Diagram	26
	3.3.5 ER Diagram	27

	3.3.6 Sequence diagram	28
	3.4 Database Design	30
	3.4.1 Logical Database Design	30
	3.4.2 Physical Database Design	30
Chapter 4	Implementation, Testing, and Maintenance	31-35
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	31
	4.2 Testing Techniques and Test Plans (According to project)	32
	4.3 Installation Instructions	33
	4.4 End User Instructions	35
Chapter 5	Results and Discussions	36-40
	5.1 User Interface Representation (of Respective Project)	36
	5.2 Brief Description of Various Modules of the system	36
	5.3 Snapshots of system with brief detail of each	37
	5.4 Back Ends Representation (Database to be used)	39
	5.5 Snapshots of Database Tables with brief description	40
Chapter 6	Summary and Conclusions	42
Chapter 7	Future scope	43
	Appendix	43
	Bibliography	44
	List of Publications (If any)	
	Reprints of publications (If any)	
Note	Results and Discussions may not be a separate chapter it may be included in main chapter	

List of Figures

<u>S. No</u>	<u>Figure</u>	<u>Page No.</u>
1.1	Objectives for Project Management System	16
2.1	Functional Requirements	20
3.1	Level 0 DFD	24
3.2	Level 1 DFD	25
3.3	Level 2 DFD	25
3.4	Activity Diagram	26
3.5	System Architecture	27
3.6	Class Diagram	28
3.7	ER Diagram (Entity-Relationship)	29
3.8	Sequence Diagram	30
3.9	Logical and Physical Database Diagram	32
4.1	Installation Instructions	36
5.1	Dashboard	41
5.2	Project View	42
5.3	Create new Project	42
5.4	Invitation to workspace	43
5.5	Users Table	44
5.6	Projects Table	44
5.7	Workspace Table	45
5.8	Task Table	45

List of Tables

<u>S. No</u>	<u>Table</u>	<u>Page No.</u>
1.1	Comparison of Existing Project Management Tools	15
1.2	SDLC Phases and Activities	17
2.1	User Roles and Permissions Matrix	19
2.2	Functional Requirements Summary	21
2.3	System Dependencies	21
3.1	Functional Modules and Responsibilities	24
4.1	PERN Stack Technology Architecture	34
4.2	Sample Test Plan Table	35
5.1	System Modules and Feature Description	41

Abbreviations

- **SRS** – Software Requirements Specification
- **UI** – User Interface
- **UX** – User Experience
- **API** – Application Programming Interface
- **DB** – Database
- **DBMS** – Database Management System
- **SQL** – Structured Query Language
- **ER** – Entity Relationship
- **DFD** – Data Flow Diagram
- **UML** – Unified Modeling Language
- **HTTP** – HyperText Transfer Protocol
- **HTTPS** – HyperText Transfer Protocol Secure
- **CRUD** – Create, Read, Update, Delete
- **SDLC** – Software Development Life Cycle
- **MVC** – Model View Controller
- **PK** – Primary Key
- **FK** – Foreign Key
- **IDE** – Integrated Development Environment
- **OS** – Operating System
- **RAM** – Random Access Memory
- **CSS** – Cascading Style Sheets
- **HTML** – Hyper Text Markup Language
- **JS** – JavaScript
- **UAT** – User Acceptance Testing
- **QA** – Quality Assurance

Notations & Symbols

- $\rightarrow$: Represents flow or direction of data/process
- $\leftrightarrow$: Represents two-way communication between components
- Δ (Delta): Represents change or difference in state/status
- Σ (Sigma): Represents summation or total (used in reports/analysis)
- $\sum$: Total number of tasks/projects (mathematical representation)
- $\in$: Element belongs to a set (e.g., user $\in$ project team)
- $\emptyset$: Null or no value
- $=$: Equality or assignment
- $\neq$: Not equal to
- $> / <$: Greater than / Less than
- $\geq / \leq$: Greater than or equal to / Less than or equal to
- $\{ \}$: Represents a set or collection (e.g., set of users or tasks)
- $[]$: Optional elements or parameters
- $()$: Grouping of expressions or parameters
- $| |$: Logical OR condition
- $\&\&$: Logical AND condition
- $!$: Logical NOT
- $/$: Separator or division
- $\%$: Percentage or modulus operation

Chapter 1: Introduction

1.1 Introduction

The Project Management System is a full-stack web application designed to streamline project planning, task assignment, and team collaboration within organizations. It provides a centralized digital platform where users can create projects, assign tasks, track progress, and collaborate effectively. The system is built using the **PERN stack** (PostgreSQL, Express.js, React.js, and Node.js), ensuring scalability, performance, and a modern user experience. By replacing traditional methods like manual spreadsheets, this application aims to reduce manual effort, improve communication, and ensure projects are completed within their deadlines.

1.2 Literature Review

Existing project management tools such as **Trello, Asana, and Jira** offer robust task tracking and collaboration features. However, these enterprise-level systems are often complex, expensive, or lack specific customization for academic or small-team environments. Recent research highlights that students and small organizations frequently struggle with the steep learning curves of these heavy tools. Furthermore, many current academic project management systems focus solely on basic CRUD operations, neglecting advanced features like intelligent notifications or scalable architectures. This project addresses these gaps by delivering a lightweight, cost-effective, and extensible solution that incorporates industry-standard background job processing and secure authentication.

Tool	Key Features	Limitations	Target Audience
Trello	Kanban boards, drag-and-drop	Limited reporting, no background jobs	Small teams
Asana	Task tracking, timelines, dashboards	Expensive for full features	Enterprise teams
Jira	Agile sprints, issue tracking	Complex setup, steep learning curve	Software teams
This System (PMS)	Role-based access, automated notifications, PERN stack	Academic scope	Students & small orgs

Table 1.1: *Comparison of Existing Project Management Tools*

1.3 Objectives

The primary goal is to develop a comprehensive system that enhances team productivity and task visibility. Specific objectives include:

- **Secure Authentication:** Developing a robust system for user registration and login using services like Clerk.
- **Project & Organization Management:** Allowing users to create and manage organizational structures and individual projects.
- **Task Lifecycle Tracking:** Enabling the creation, assignment, and real-time status monitoring of tasks.
- **Automated Notifications:** Implementing email reminders and updates for deadlines and assignments using background job processing.
- **User-Centric Design:** Creating a responsive interface accessible across desktops, laptops, and mobile devices.
- **Real-World Deployment:** Deploying the application on cloud platforms like Vercel to ensure availability and scalability.

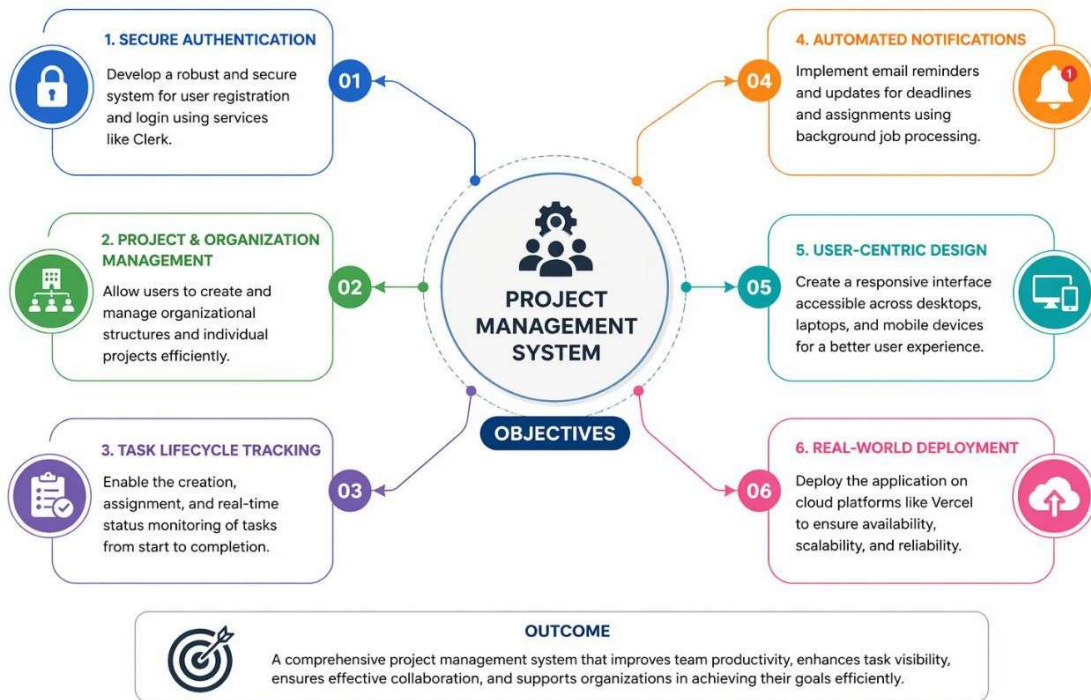


Fig 1.1: Objectives for Project Management System

1.4 Significance

This project is significant as it provides a production-level structure that simplifies project tracking for student teams and small organizations. It offers an industry-level learning experience by implementing a clean folder structure, secure API design, and cloud database management. The system's use of modern technologies—such as background job processing for automated emails—makes it a unique and scalable alternative to basic task-tracking applications.

1.5 Research Design (Methodology)

The project follows a structured **Software Development Life Cycle (SDLC)**:

1. **Requirement Analysis:** Identifying system needs and user requirements.
2. **System Design:** Creating the database schema (ER diagrams), API structures, and UI wireframes.
3. **Implementation:** Developing the frontend with React.js and the backend with Node.js and Express.js, using PostgreSQL for relational data storage.
4. **Integration:** Incorporating Clerk for authentication and Inngest for background automated notifications.
5. **Testing & Deployment:** Conducting unit and integration testing followed by deployment on cloud hosting services.

SDLC Phase	Key Activities	Output / Deliverable
Requirement Analysis	Stakeholder interviews, use-case identification	SRS Document
System Design	ER diagrams, UI wireframes, API design	Design Document
Implementation	Frontend (React), Backend (Node/Express), DB (PostgreSQL)	Working Codebase
Integration	Clerk auth, Inngest notifications, API integration	Integrated System
Testing & Deployment	Unit, integration, UAT testing; cloud deployment	Deployed Application

Table 1.2: *SDLC Phases and Activities*

1.6 Source of Data

The system relies on various data sources to function:

- **Primary Data:** User-provided information such as registration details, project descriptions, task assignments, and feedback comments.
- **Secondary Data:** Architectural and design insights gathered from established software engineering principles and documentation for the React, Node.js, and PostgreSQL frameworks.
- **Relational Database:** A structured relational model using PostgreSQL to manage data consistency and integrity across user, task, and project tables.

1.7 Chapter Scheme

The documentation is structured to provide a logical flow of the project's development:

- **Chapter 1 (Introduction):** Overview, purpose, and objectives.
- **Chapter 2 (Overall Description):** Product perspective, functions, and operating environment.
- **Chapter 3 (System Features):** Detailed functional and non-functional requirements.
- **Chapter 4 (External Interface Requirements):** Descriptions of user, hardware, and software interfaces.
- **Chapter 5 (System Design & UML):** Visual representations including Use Case, Class, and Sequence diagrams.
- **Chapter 6 (Database Design):** ER diagrams and detailed table structures.
- **Chapter 7 (Testing & Project Plan):** Testing strategies, risk analysis, and the development timeline.

Chapter 2: Requirements Specification

2.1 User Characteristics

The system identifies four primary user classes, each with specific roles and responsibilities:

- **Admin:** Manages the overall system, including users, projects, and general settings. They possess full access and control over all functionalities.
- **Project Manager:** Responsible for creating and managing specific projects. They assign tasks to team members and monitor real-time progress.
- **Team Member:** Focuses on completing assigned tasks. They are responsible for updating task statuses and collaborating with other members.
- **Client/User (Optional):** Can view project progress and status to stay informed. They provide feedback when requested by the team.

Permission	Admin	Project Manager	Team Member	Client
Manage Users	Yes	No	No	No
Create Project	Yes	Yes	No	No
Edit Project	Yes	Yes	No	No
Assign Tasks	Yes	Yes	No	No
Update Task Status	Yes	Yes	Yes	No
View All Projects	Yes	Yes	Own	View Only
Upload Documents	Yes	Yes	Yes	No
Comment on Tasks	Yes	Yes	Yes	Yes
View Reports/Dashboard	Yes	Yes	Yes	Limited
Manage System Settings	Yes	No	No	No

Table 2.1: *User Roles and Permissions Matrix*

2.2 Functional Requirements

The core functionalities of the system are categorized as follows:

- **User Authentication:** The system shall allow users to register with valid details, authenticate via email/password, and provide role-based access control.

- **Project Management:** Users must be able to create, edit, delete, and view all projects within their organization.
- **Task Management:** The system shall support task creation, assignment to users, status updates, and deadline tracking.
- **Team Management:** Capabilities include adding or removing team members and assigning specific roles to them.
- **File & Document Management:** The system shall allow secure file uploads and provide team-wide access to stored documents.
- **Communication & Notifications:** Users should be able to comment on tasks and receive automated alerts for assignments or upcoming deadlines.

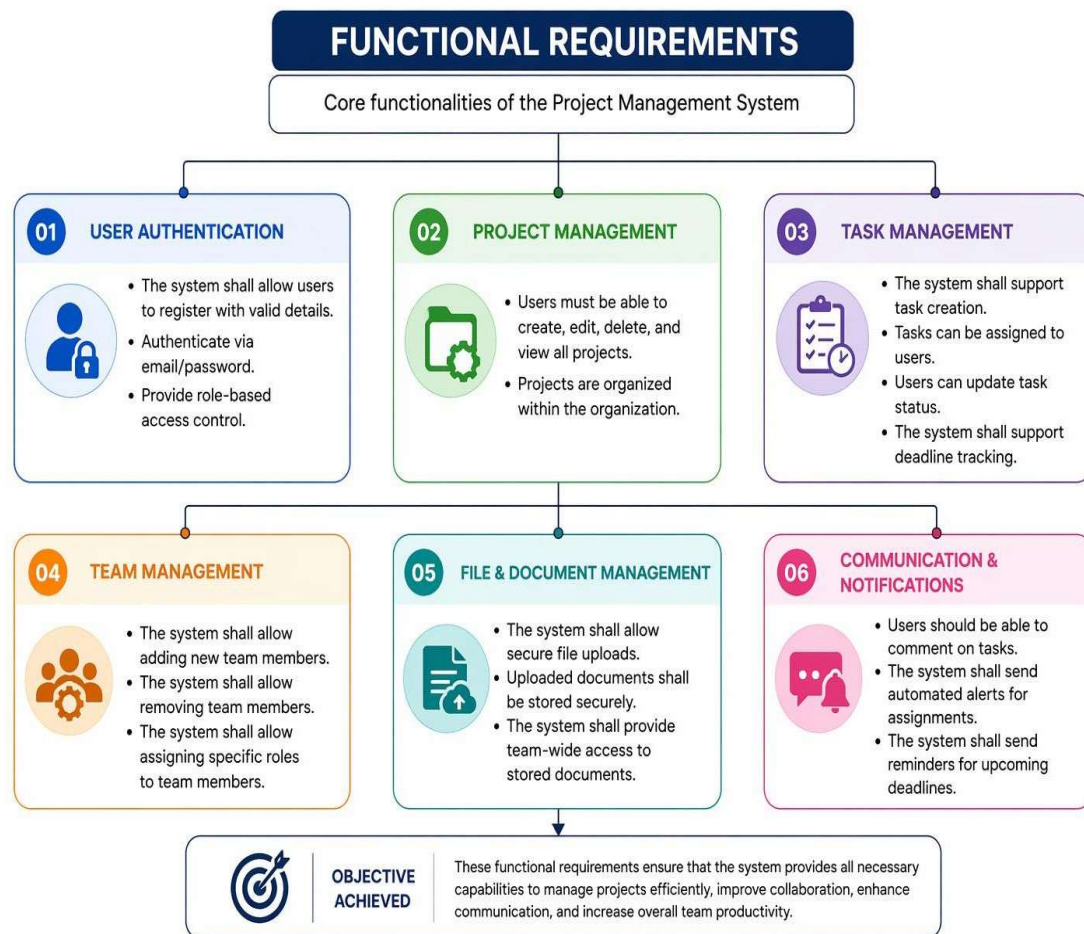


Fig 2.1: Functional Requirements

Req. ID	Functional Area	Description	Priority
FR-01	User Authentication	Register, login, role-based access via Clerk	High
FR-02	Project Management	Create, edit, delete, view projects	High
FR-03	Task Management	Create, assign, track, update task status	High
FR-04	Team Management	Add/remove members, assign roles	Medium
FR-05	File Management	Secure upload and team-wide access to documents	Medium
FR-06	Communication	Task commenting and automated notifications	High

Table 2.2: *Functional Requirements Summary*

2.3 Dependencies

The system's successful operation relies on several external factors:

- **Web Server:** Hosting is required for the application to be accessible online.
- **Database System:** Relies on a robust database (PostgreSQL/MySQL/MongoDB) for persistent data storage.
- **Connectivity:** A stable internet connection is essential for real-time communication between the client and server.
- **Development Frameworks:** The application depends on specific tools like Node.js, React, and Express for its core logic.

Dependency	Type	Purpose	Availability
PostgreSQL / MySQL / MongoDB	Database	Persistent data storage and management	Cloud / Local
Node.js & Express.js	Backend Framework	Server-side API logic and routing	Open-source
React.js	Frontend Library	Building the user interface	Open-source
Clerk	Auth Service	User authentication and session management	SaaS / Cloud
Inngest	Job Processor	Background job queue and notifications	SaaS / Cloud
Web Hosting (Vercel/Render)	Hosting	Deployment of frontend and backend	Cloud SaaS

Table 2.3: *System Dependencies*

2.4 Performance Requirements

To ensure a high-quality user experience, the system must meet these benchmarks:

- **Response Time:** The system should respond to user actions within 2–3 seconds.
- **Concurrency:** It must support multiple simultaneous users without any performance degradation.
- **Data Efficiency:** The system must handle large volumes of project and task data efficiently.
- **Real-time Processing:** Status updates and notifications must be processed quickly to maintain data accuracy.

2.5 Hardware Requirements

The application is designed to be lightweight and accessible:

- **Supported Devices:** Can run on desktops, laptops.
- **RAM:** A minimum of 4 GB RAM is recommended for optimal performance.
- **Input Devices:** Requires standard input components like a keyboard, mouse, or touchscreen.
- **Infrastructure:** No specialized or proprietary hardware is necessary.

2.6 Constraints & Assumptions

Constraints:

- **Platform:** The system must remain strictly web-based and accessible via standard browsers like Chrome or Firefox.
- **Security:** Must implement secure authentication and ensure data encryption (HTTPS).
- **Technical Limits:** Development is constrained by the capabilities of the chosen PERN stack (PostgreSQL, Express, React, Node).

Assumptions:

- Users possess a basic understanding of how to navigate web applications.
- All data provided by users during registration and project creation will be accurate.
- The system will be accessed through modern, updated web browser.

Chapter 3: Design

3.1 Algorithm Descriptions

The Project Management System employs specific logical procedures to handle critical background tasks and data operations:

- **Authentication & Session Algorithm:** The system leverages **Clerk** for secure user identity management. When a user attempts to log in, the system sends credentials to the authentication server, which returns a secure JWT (JSON Web Token). This token is verified on every subsequent API request to ensure that the user has the correct permissions for the requested resource.
- **Notification Scheduling (Background Processing):** To avoid slowing down the main application thread, the system uses **Inngest** for background job processing.
 1. An event (e.g., "Task Assigned") triggers a background job.
 2. The job enters a queue and is processed independently of the user interface.
 3. The algorithm fetches the assignee's email and project details to send an automated reminder via a cloud email service.
- **State Management Logic:** On the frontend, **React** utilizes component-level and global state management to ensure that UI components (like the task board) update immediately when a user changes a task status, followed by an asynchronous update to the **PostgreSQL** database.

3.2 Function-Oriented Design

The system is decomposed into specialized functional modules that communicate through a **RESTful API**:

- **User Management Module:** Handles user registration, role assignment (Admin, Manager, Member), and profile updates.
- **Core Project Engine:** Manages the lifecycle of projects, including initialization, setting timelines, and archiving completed work.
- **Task Logic Module:** Controls the granular work items, including subtask hierarchies, deadline tracking, and status transitions.
- **Collaboration Engine:** Facilitates team interaction through a commenting system and secure document sharing.

Module	Key Functions	Input	Output
User Management	Register, login, role assign, profile update	User credentials, role data	Auth token, user profile
Core Project Engine	Create, edit, archive projects	Project metadata	Project record in DB
Task Logic Module	Create, assign, track, update tasks	Task data, assignee ID	Task status update
Collaboration Engine	Comments, file sharing	Comment text, file uploads	Stored comments, file URLs
Notification Module	Email alerts, in-app notifications	Task/deadline events	Email & notification record

Table 3.1: *Functional Modules and Responsibilities*

3.3 System Design (Visual Architectures)

3.3.1 Data Flow Diagrams (DFDs)

- **Level 0 (Context Diagram):** Positions the system as a central hub where the **Admin** manages users, **Managers** input project data, and **Team Members** update task outputs.
- **Level 1 DFD:** Details the flow from the **Frontend (Presentation Layer)** to the **Backend (Application Layer)** and finally to the **Database (Data Layer)**. It highlights how data like "Task Status" moves from a user's browser to the server for validation and storage.



Fig 3.1 : *Level 0 DFD*

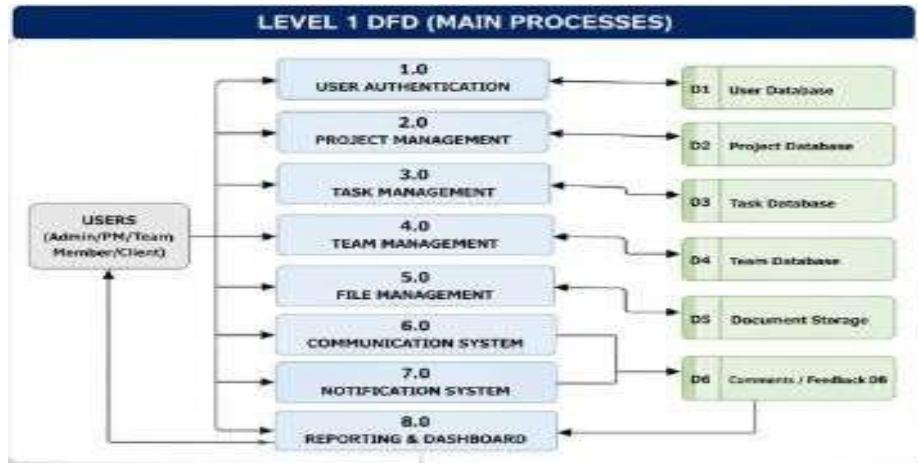


Fig 3.2: Level 1 DFD

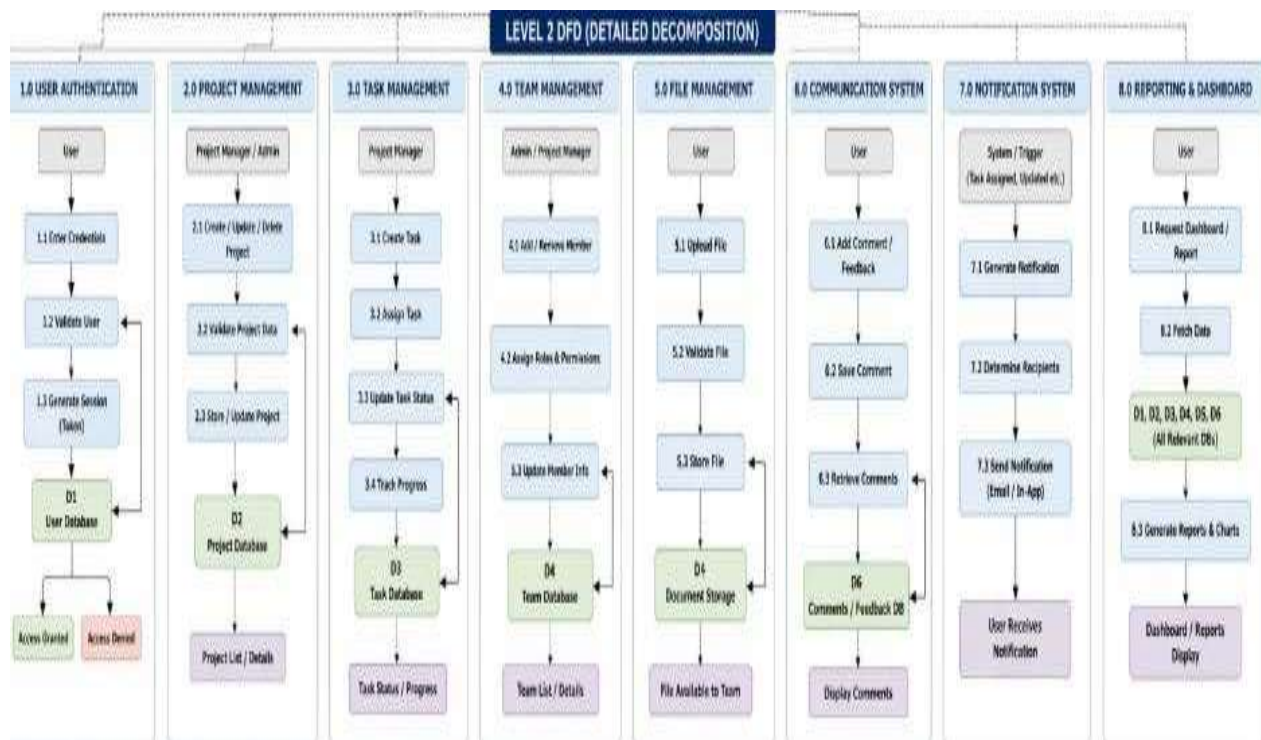


Fig 3.3: Level 2 DFD

3.3.2 Activity Diagram

An **Activity Diagram** is a type of UML (Unified Modeling Language) diagram used to represent the **flow of activities or actions in a system**. It shows how different tasks are performed step-by-step, including decisions, loops, and parallel processes.

Describes the dynamic behavior of the system, such as the **Project Creation Workflow**:

1. Manager initiates the "New Project" action.
2. System presents a form for title, description, and deadlines.
3. Manager submits data; the system validates input.
4. If valid, the system creates the database record and redirects to the dashboard; if invalid, it displays an error.

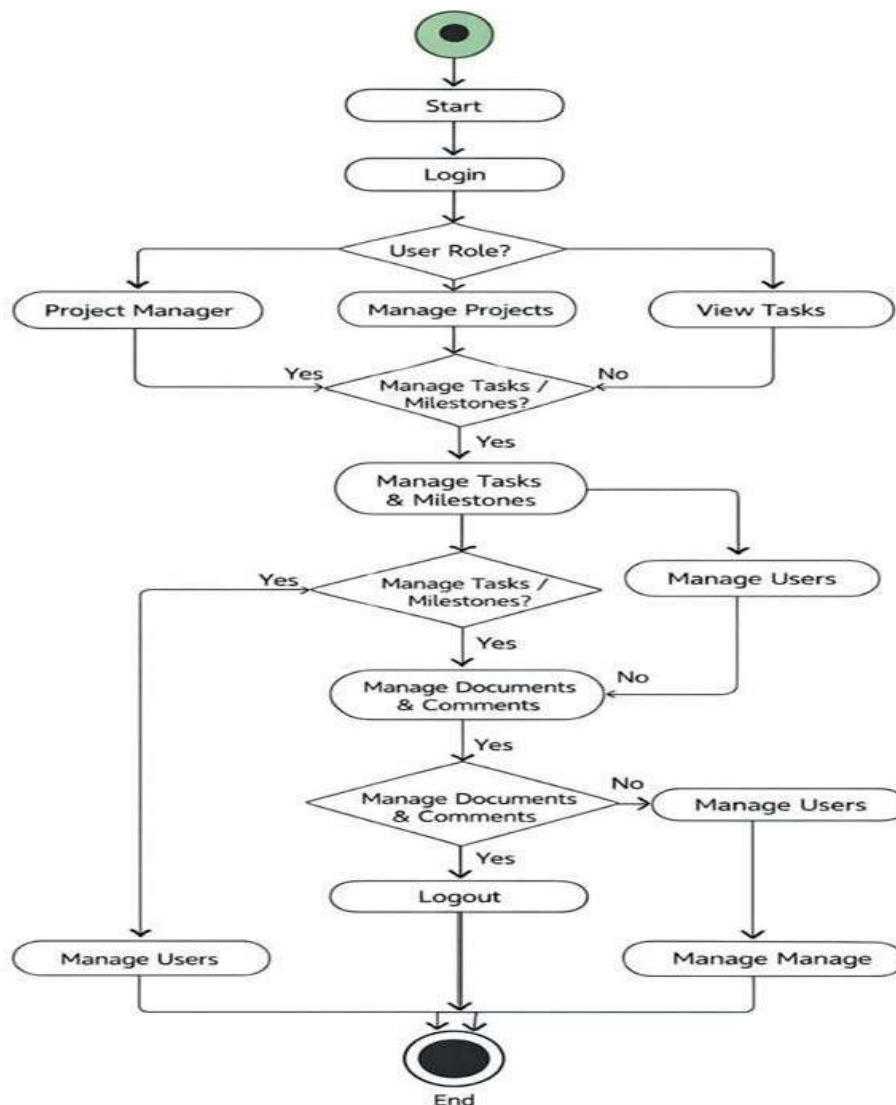


Fig 3.4: Activity Diagram

3.3.3 System Architecture (3-Tier Model) The system follows a standard industry architecture to ensure separation of concerns:

- **Presentation Layer:** Built with **React.js**, providing a responsive dashboard for desktop and mobile devices.
- **Application Layer:** Powered by **Node.js** and **Express.js**, it processes business logic, manages authentication, and handles API requests.
- **Data Layer:** A **PostgreSQL** relational database that ensures data integrity and manages complex relationships between users, tasks, and projects.

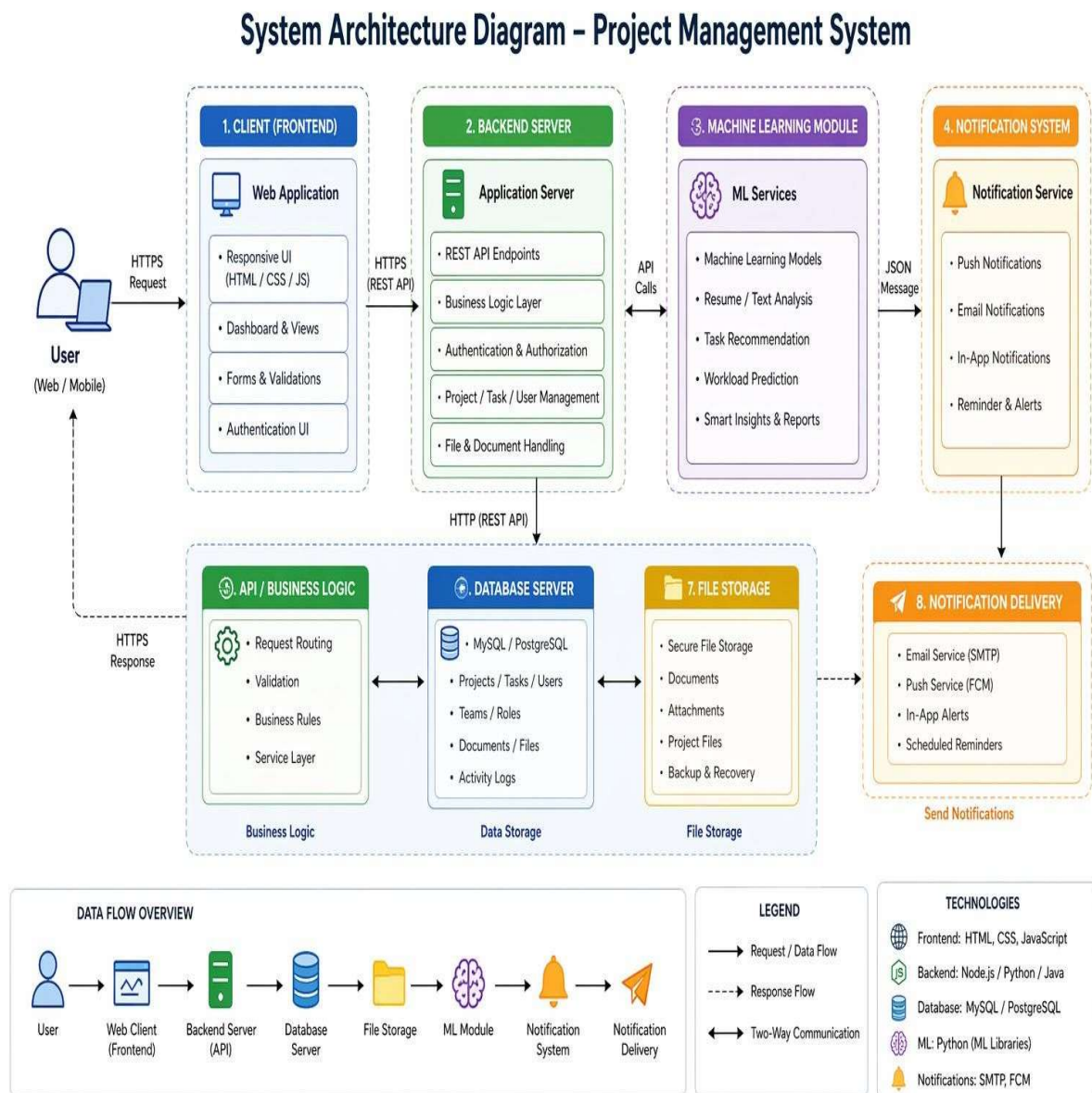


Fig 3.5: System Architecture

3.3.4 Class Diagram Defines the static structure of the code, including:

- **User Class:** Attributes include user_id, email, and role; methods include login() and updateProfile().
- **Project Class:** Attributes include title and status; methods include createProject() and getProjectDetails().
- **Task Class:** Attributes include due_date and priority; methods include updateStatus().

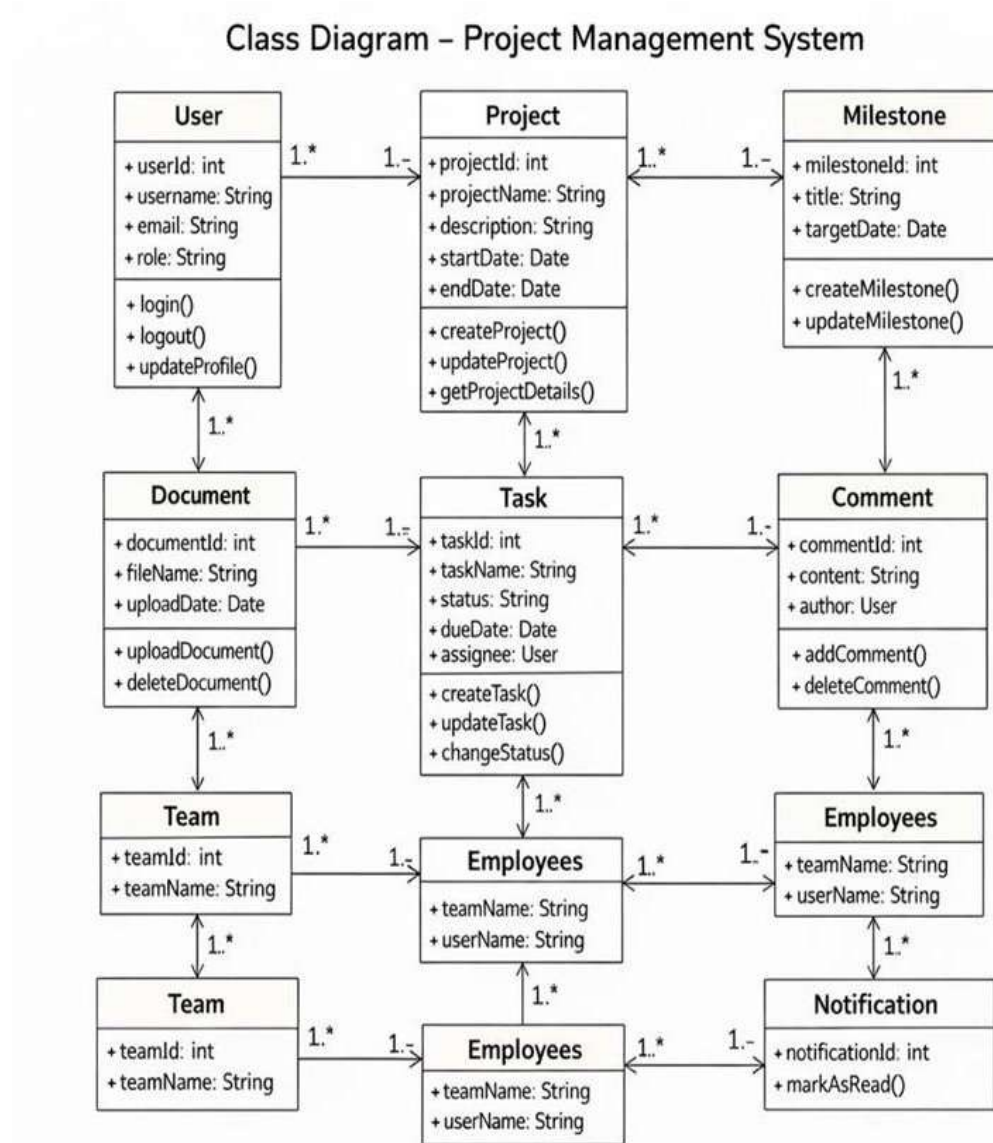


Fig 3.6: Class Diagram

3.3.5 ER Diagram (Entity-Relationship) Maps the database relationships:

- **Users to Projects:** A many-to-many relationship where users can belong to multiple projects and projects can have many users.
- **Projects to Tasks:** A one-to-many relationship where one project contains multiple tasks.
- **Tasks to Subtasks:** A one-to-many hierarchy for detailed work tracking.

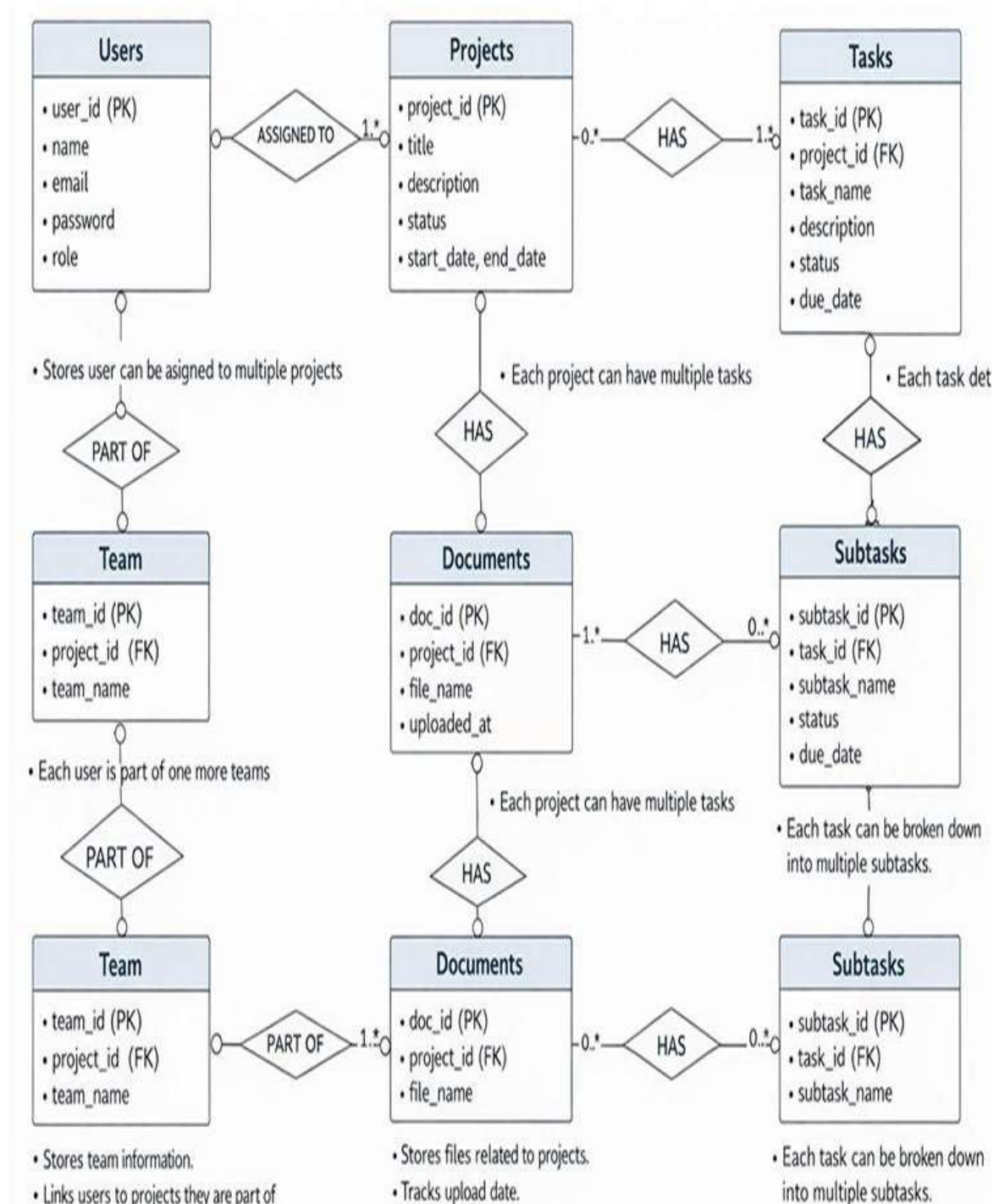


Fig 3.7: ER Diagram (Entity-Relationship)

3.3.6 Sequence Diagram Illustrates the time-ordered interaction for a **Task Assignment**:

1. **Manager Frontend**: Selects member and creates task.
2. **Frontend Backend**: POST request with task payload.
3. **Backend Database**: Insert task record.
4. **Backend Inngest**: Trigger background notification.
5. **Inngest Member**: Sends email notification.

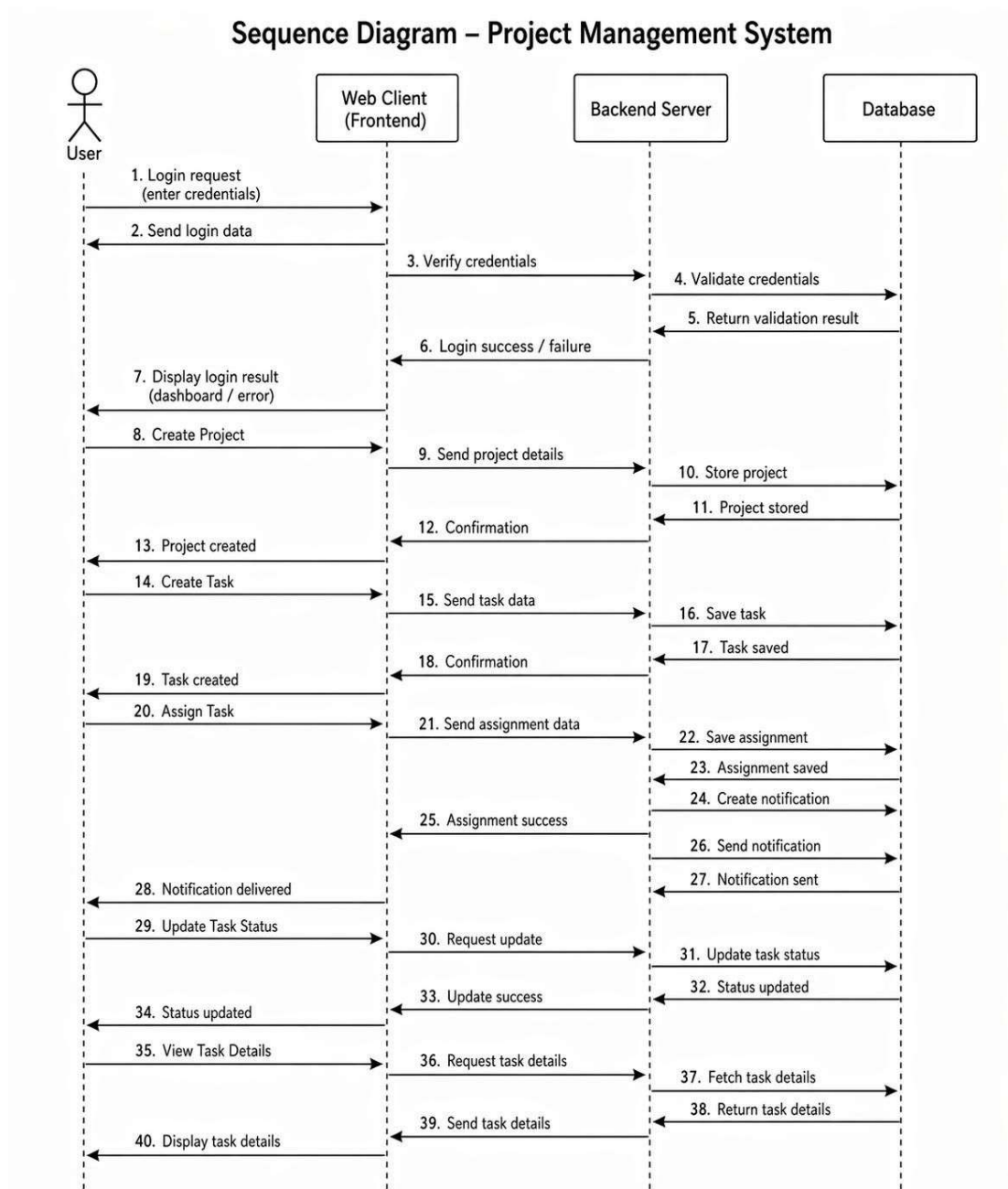


Fig 3.8: *Sequence Diagram*

3.4 Database Design

3.4.1 Logical Database Design The schema is organized into normalized tables to prevent data redundancy:

- **Users Table:** Primary storage for account details and roles (user_id, name, email, role).
- **Projects Table:** Stores project-specific metadata (project_id, title, description, status).
- **Tasks & Subtasks:** Linked via foreign keys to their respective parent projects and tasks to maintain a clear hierarchy.
- **Documents & Team Tables:** Manage file associations and project-level group memberships.

3.4.2 Physical Database Design

- **DBMS Selection:** PostgreSQL was chosen for its support of complex relational queries and robust data integrity features.
- **Integrity Constraints:** The design uses **Primary Keys (PK)** for unique identification and **Foreign Keys (FK)** to enforce referential integrity.
- **Deployment:** The database is hosted on a **Cloud Database platform** to ensure 24/7 availability, automatic backups, and scalability as the data volume grows.

3.4 DATABASE DESIGN – PROJECT MANAGEMENT SYSTEM

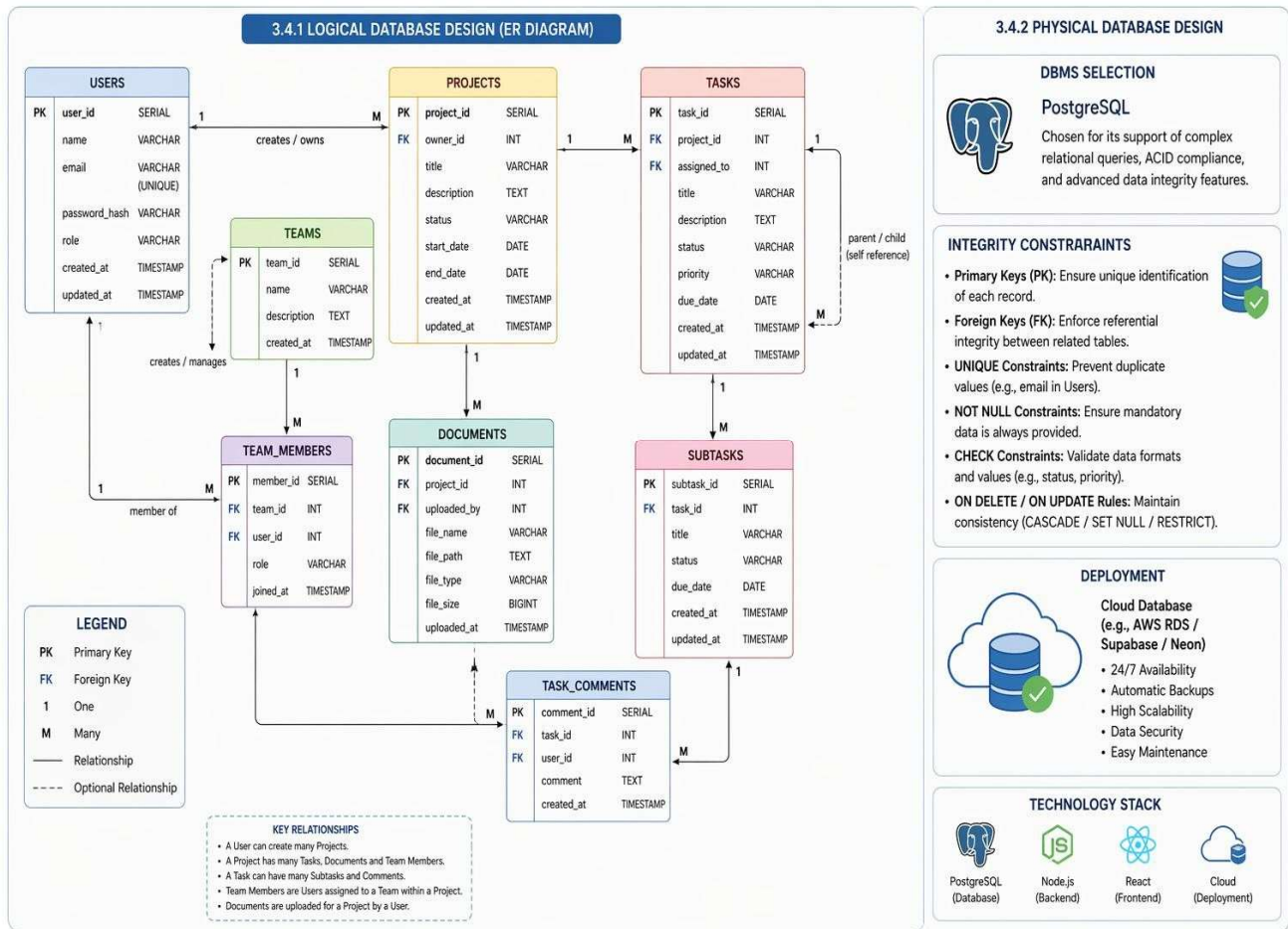


Fig 3.9: Logical and Physical Database Diagram

Chapter 4: Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies

The implementation phase utilizes a modern, industry-standard stack known as the **PERN stack**, designed for high performance and scalability.

- **Programming Languages & Frameworks:**
 - **JavaScript (ES6+):** The primary language used for both frontend and backend logic, ensuring a unified development experience.
 - **React.js (Frontend):** A powerful library for building dynamic user interfaces. It allows for component-based architecture, making the UI reusable and easy to maintain.
 - **Node.js & Express.js (Backend):** Node.js provides a runtime environment for executing JavaScript on the server, while Express.js acts as a minimalist web framework to handle API routing and middleware.
- **Database:**
 - **PostgreSQL:** An advanced, open-source relational database used to store complex data relationships between users, projects, and tasks with high integrity.
- **Integrated Development Environment (IDE):**
 - **Visual Studio Code (VS Code):** Chosen for its extensive support for JavaScript, built-in terminal, and version control integration (Git).
- **Tools & Technologies:**
 - **Clerk:** Used for secure, production-level authentication and user session management.
 - **Inngest:** A background job processing tool used to handle asynchronous events like automated email notifications and deadline reminders.
 - **Postman:** Utilized for testing API endpoints during development to ensure request-response cycles are functioning correctly.
 - **Git & GitHub:** Used for version control, allowing team members to collaborate and track changes effectively.
 - **Vercel/Render:** Targeted platforms for online deployment and hosting.

Tool / Technology	Category	Purpose
Visual Studio Code	IDE	Primary code editor with JavaScript/React extensions
Clerk	Auth Service	Production-level authentication, session management, social login
Inngest	Background Jobs	Async event processing; email reminders, deadline alerts
Postman	API Testing	Testing REST API endpoints during development
Git & GitHub	Version Control	Source code management and team collaboration
Vercel	Frontend Hosting	Deployment platform for React.js frontend
Render	Backend Hosting	Deployment platform for Node.js/Express.js backend
pgAdmin / TablePlus	DB Management	Visualize and manage PostgreSQL database tables
Prisma ORM	ORM Tool	Type-safe database queries and schema migrations

Table 4.1: *PERN Stack Technology Architecture*

4.2 Testing Techniques and Test Plans

To ensure the reliability and security of the Project Management System, a multi-layered testing strategy is adopted.

- **Unit Testing:** Individual functions and components (e.g., a "Create Task" function or a "Login" form) are tested in isolation to ensure they produce the correct output for a given input.
- **Integration Testing:** Ensures that different modules, such as the Frontend-to-API communication or the Backend-to-Database connection, work together seamlessly.
- **System Testing:** The entire application is tested as a whole against the functional requirements defined in the SRS to ensure all features work in a live environment.
- **User Acceptance Testing (UAT):** Conducted to ensure the system meets the end-user's needs and provides an intuitive experience.

Test Case ID	Feature	Input	Expected Output
TC-01	User Login	Valid email/password	Redirect to Dashboard
TC-02	Task Creation	Task name & deadline	Task appears in "To-Do" list
TC-03	Input Validation	Empty task name	Display "Task name is required"
TC-04	Notification	Assigning a task	Team member receives an email

Table 4.2: Sample Test Plan Table

4.3 Installation Instructions

To set up the project locally for development or maintenance, follow these steps:

1. **Prerequisites:** Install **Node.js** (v18 or higher) and **PostgreSQL** on your local machine.
2. **Clone Repository:** `git clone <repository-url>`
3. **Frontend Setup:**
 - Navigate to the client folder: `cd client`
 - Install dependencies: `npm install`
 - Start development server: `npm run dev`
4. **Backend Setup:**
 - Navigate to the server folder: `cd server`
 - Install dependencies: `npm install`
 - Configure Environment Variables: Create a `.env` file and add your `DATABASE_URL`, `CLERK_SECRET_KEY`, and `INNGEST_EVENT_KEY`.
 - Run migrations: `npx prisma db push` (if using Prisma) or run SQL scripts.
 - Start server: `npm start`

4.3 INSTALLATION INSTRUCTIONS

Follow the steps below to set up the Project Management System locally.

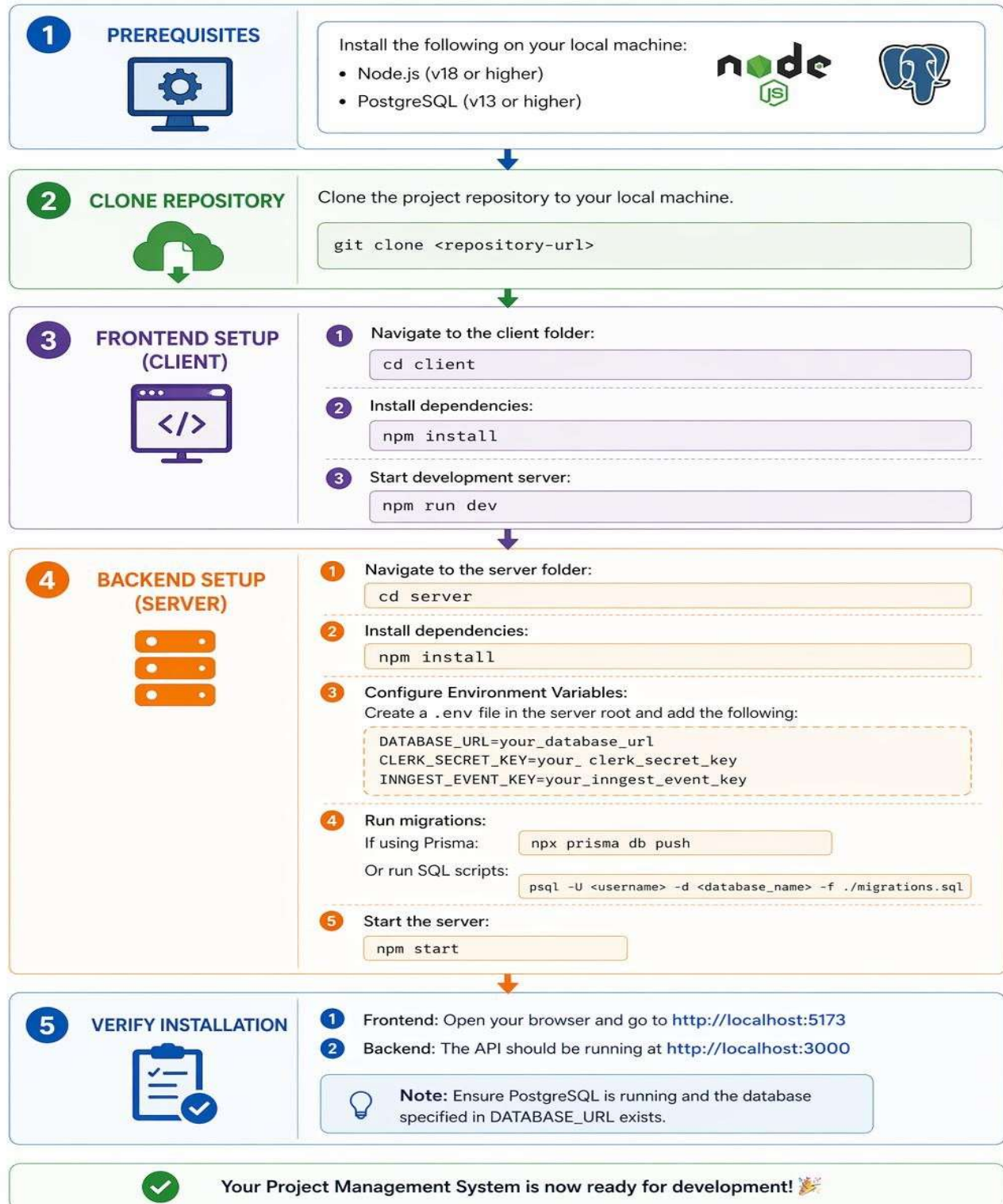


Fig 4.1: Installation Instructions

4.4 End User Instructions

This section serves as a brief manual for the users of the Project Management System:

- **Getting Started:** Visit the application URL and click on "Sign Up" to create an account. You will receive a verification email via Clerk.
- **Creating an Organization:** Upon first login, users can create or join an organization to start collaborating with a team.
- **Managing Projects:** Click the "New Project" button on the dashboard. Provide a title, description, and select team members.
- **Working with Tasks:** Within a project, use the "Add Task" button. You can drag and drop tasks between "To-Do," "In Progress," and "Done" columns to update their status.
- **Collaborating:** Click on any task to open the detail view, where you can leave comments for your team or upload relevant documents.
- **Monitoring Progress:** Use the Dashboard's visual charts to track how many tasks are completed versus pending as the deadline approaches.

Chapter 5: Results and Discussions

5.1 User Interface Representation

The User Interface (UI) of the Project Management System serves as the primary bridge between complex backend logic and user productivity. It is developed using React.js for a Single Page Application (SPA) experience, ensuring that navigation between projects and tasks is instantaneous without full-page reloads.

- **Design Philosophy:** The interface follows the Human Interface Guidelines, prioritizing clarity and hierarchy. High-contrast typography and a neutral color palette are used to reduce eye strain during long working hours.
- **Component Architecture:** The UI is composed of reusable components such as TaskCard, Project Header, and User Avatar. This ensures visual consistency across the entire application.
- **Stateful Feedback:** Every user interaction provides immediate visual feedback. For example, while a task is being moved in the Kanban board, a "ghost" element indicates the drop zone, and a loading spinner appears during the database synchronization phase.
- **Dashboard Analytics:** The main landing area for logged-in users features dynamic widgets. These include "Burn-down Charts" to visualize progress over time and "Activity Feeds" to show recent comments or status changes across the organization.

5.2 Brief Description of Various Modules of the System

The system is architected into five distinct, interconnected modules:

1. **Identity and Access Module (Clerk Integration):** This module handles the most sensitive part of the application. It supports social logins, email verification, and session persistence. It ensures that data is isolated—users can only see projects belonging to their specific organization.
2. **Organization Workspace Module:** This acts as the "root" container. Users can create multiple organizations (e.g., "Development Team" and "Marketing Team"). Each workspace has its own set of members and permissions.
3. **Project Engine:** This module manages the metadata of work. It handles the creation of project timelines, descriptions, and the linking of specific team members to specific projects.
4. **Task & Sprint Management Module:** Utilizing a relational model, this module handles the lifecycle of a task from "Backlog" to "Completed." It includes logic for sub-tasks, priority tags (Low, Medium, High, Urgent), and deadline tracking.
5. **Automated Workflow Module (Inngest):** A sophisticated background layer that automates repetitive tasks. If a task is not updated within 24 hours of its deadline, this module automatically triggers a reminder.

Module	Technology	Key Features	User Role(s)
Identity & Access	Clerk	Social login, email verify, session management, org isolation	All Users
Organization Workspace	React + PostgreSQL	Multi-org support, member management, permission levels	Admin, Manager
Project Engine	Express + PostgreSQL	Project creation, timeline, member linking, archiving	Admin, Manager
Task & Sprint Management	React + PostgreSQL	Kanban board, drag-drop, priority tags, subtasks, deadlines	All Users
Automated Workflow	Inngest	Background jobs, email reminders, 24-hr deadline alerts	System (automated)

Table 5.1: *System Modules and Feature Description*

5.3 Snapshots of System with Brief Detail

- Snapshot 1: Dashboard- The Dashboard serves as the central overview of the Project Management System, providing users with a summarized view of all activities and progress. It displays key information such as active projects, pending tasks, completed tasks, upcoming deadlines, and recent notifications.

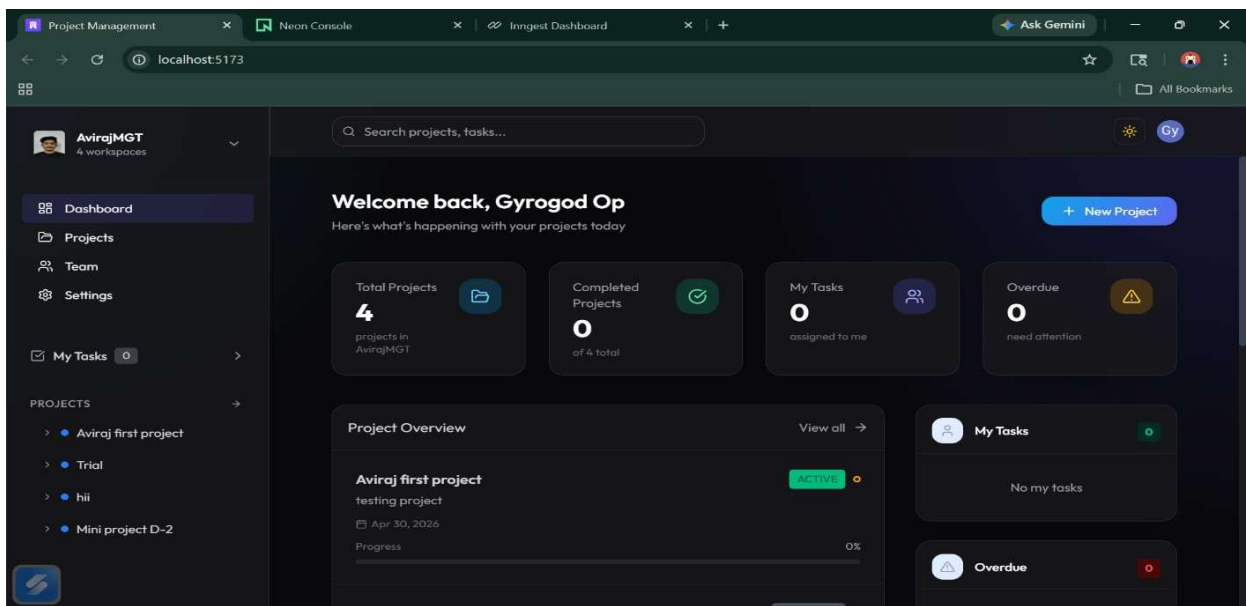


Fig 5.1 *Dashboard*

- Snapshot 2: Project view – Manage and track your Projects.
The Project View provides a detailed interface for managing and tracking a specific project. It includes information such as project title, description, team members, deadlines, and current status.

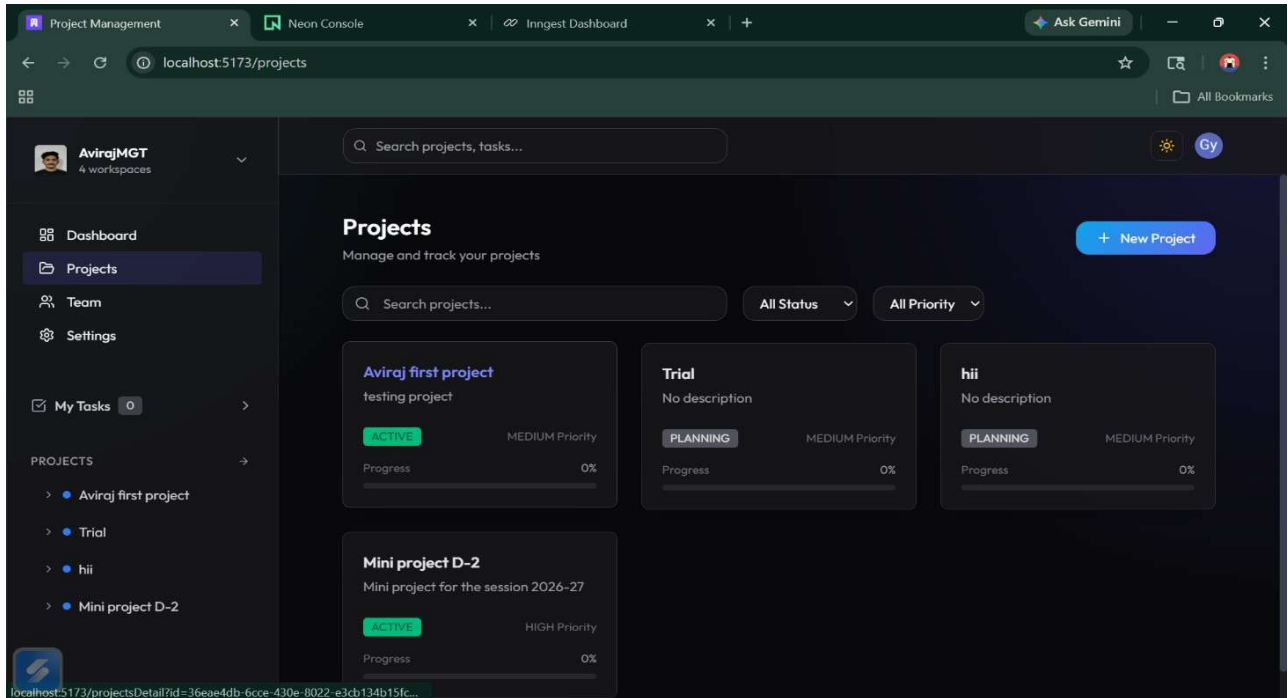


Fig 5.2 Project View

- Snapshot 3: Create new Projects.
The Create New Project feature allows users to initiate a new project by entering essential details such as project name, description, start date, and end date. Users can also define project goals, assign team members, and set initial tasks during the creation process.

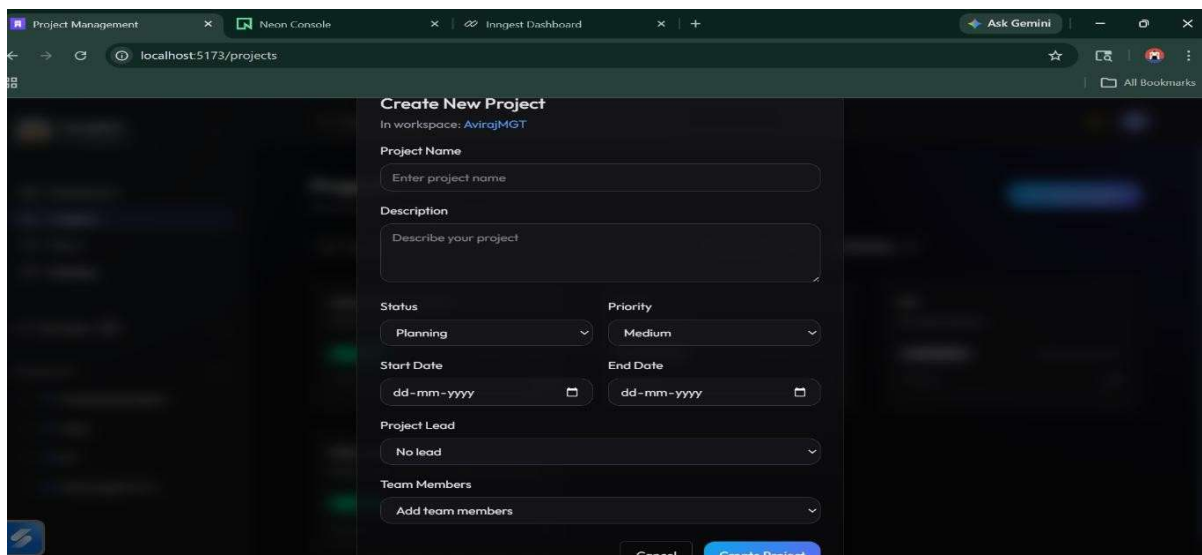


Fig 5.3 Create new Project

- Snapshot 4: Invitation for team members- The Invitation to Workspace feature enables seamless collaboration by allowing users to invite team members to join a project or organization. Invitations can be sent via email, where the recipient receives a link to join the workspace.

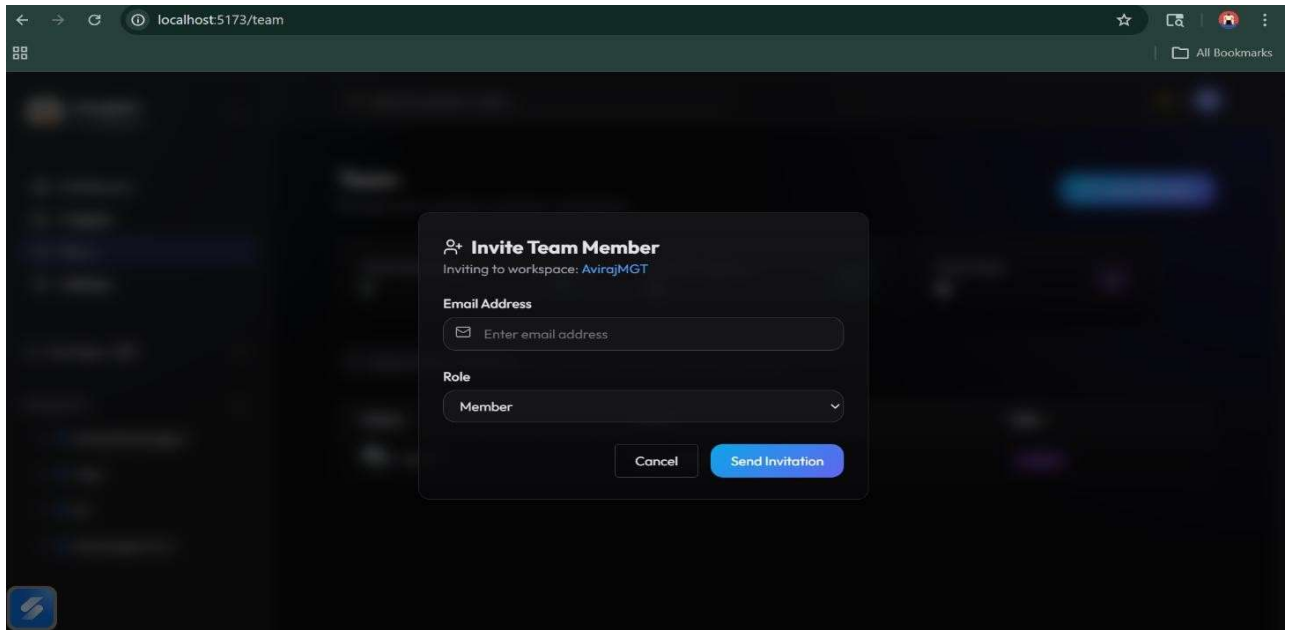


Fig 5.4 *Invitation to workspace*

5.4 Back End Representation (Database Design)

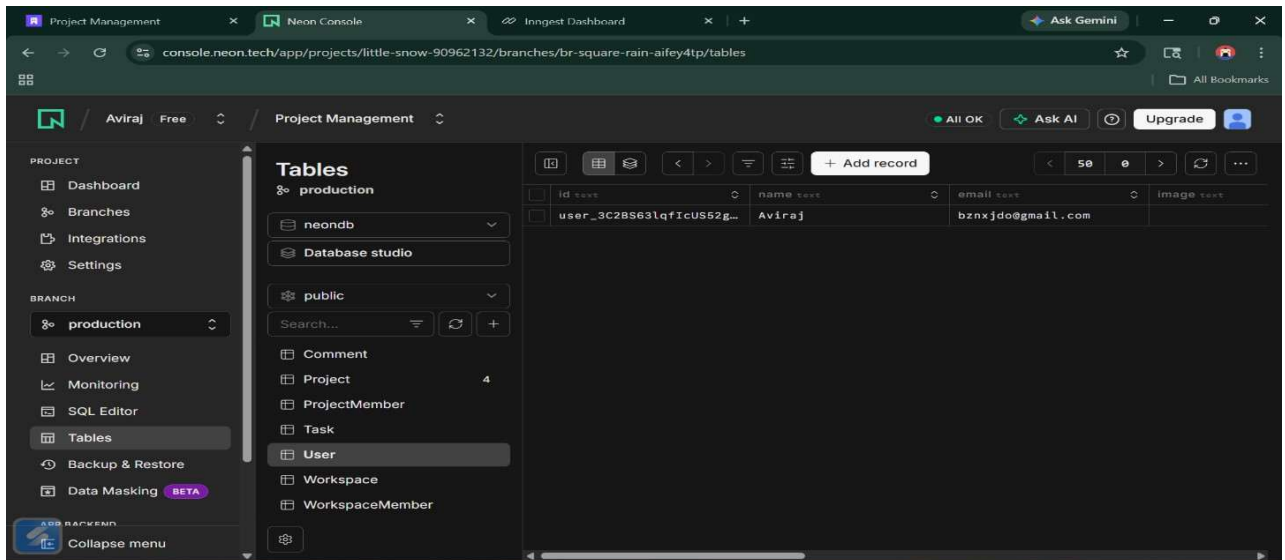
The backend is built using the PERN stack, with PostgreSQL serving as the core relational engine. The choice of a relational database was critical due to the complex associations between entities.

- **Relational Logic:** The database uses a schema-first approach. For instance, the Task table has a mandatory project id foreign key. If a project is deleted, a "Cascade Delete" or "Soft Delete" logic ensures the database remains clean and consistent.
- **Data Security:** All sensitive data at the database level is protected. While Clerk handles passwords, the PostgreSQL database stores User IDs and project data using encrypted connections (SSL/TLS) to prevent data intercept during transmission.
- **Query Optimization:** As the project grows, the backend uses indexing on frequently searched columns like status and due date to ensure that the dashboard loads quickly even with thousands of tasks.

5.5 Snapshots of Database Tables with Brief Description

1. users Table:

- *Columns:* id (PK), clerk id, email, name, image url.
- *Description:* Maps the external authentication ID to internal application data.

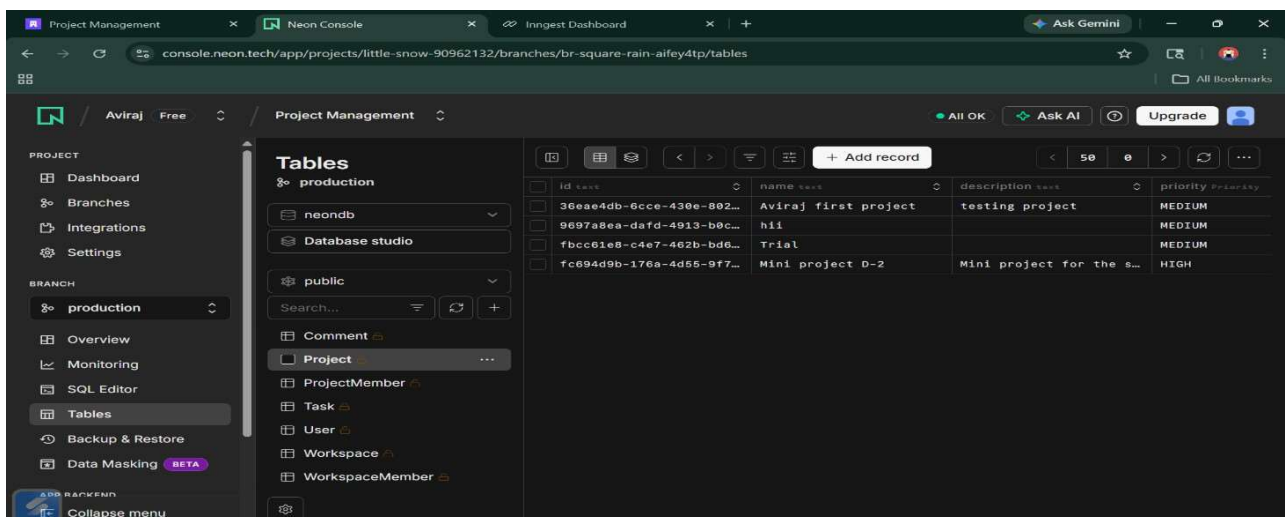


id	name	email	image
user_3C2B8631qfIcUS52g...	Aviraj	bznxjdo@gmail.com	

Fig 5.5 users table

2. projects Table:

- *Columns:* id (PK), name, description, priority.
- *Description:* Stores the high-level grouping of tasks. Every project must belong to an organization.

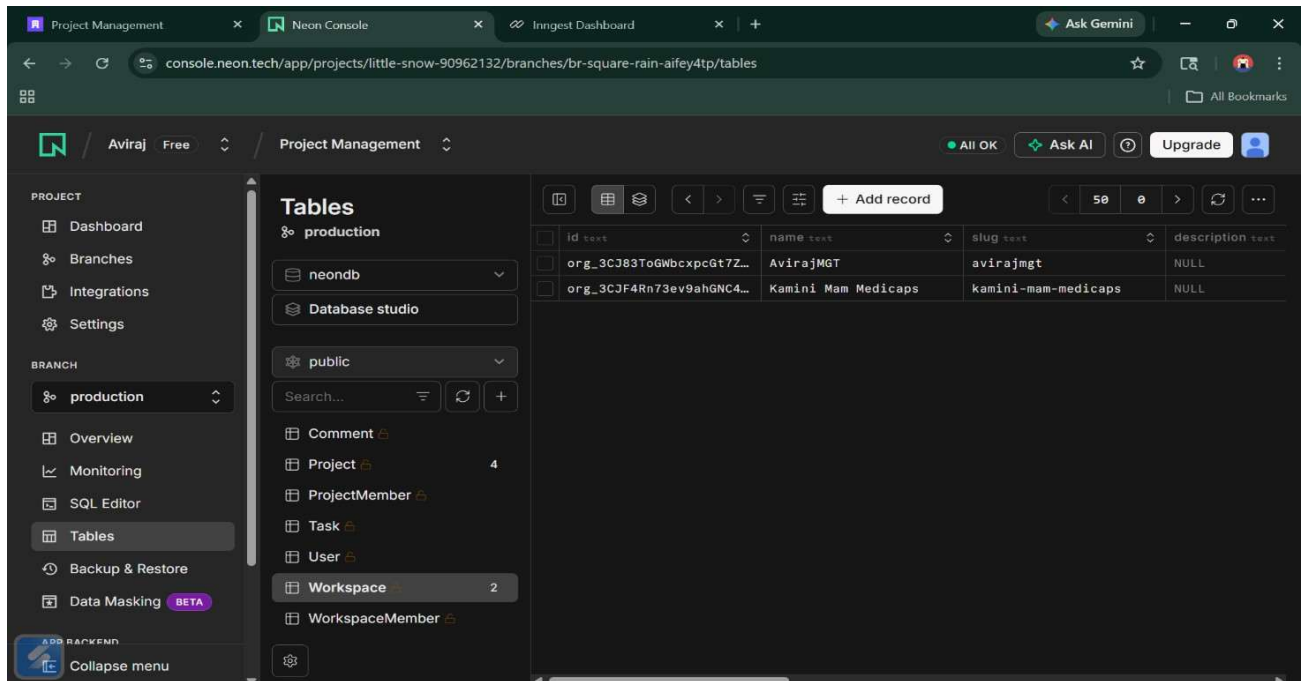


id	name	description	priority
36eae4db-6cce-430e-802...	Aviraj first project	testing project	MEDIUM
9697a8ea-da1d-4913-b0c...	hii		MEDIUM
fbcc61e8-c4e7-402b-bd6...	Trial		MEDIUM
fc694d9b-176a-4d55-9f7...	Mini project D-2	Mini project for the s...	HIGH

Fig 5.6 projects table

3. Workspace Table:

- *Columns:* id (PK), name, description, slug.
- *Description:* Stores the workspace information



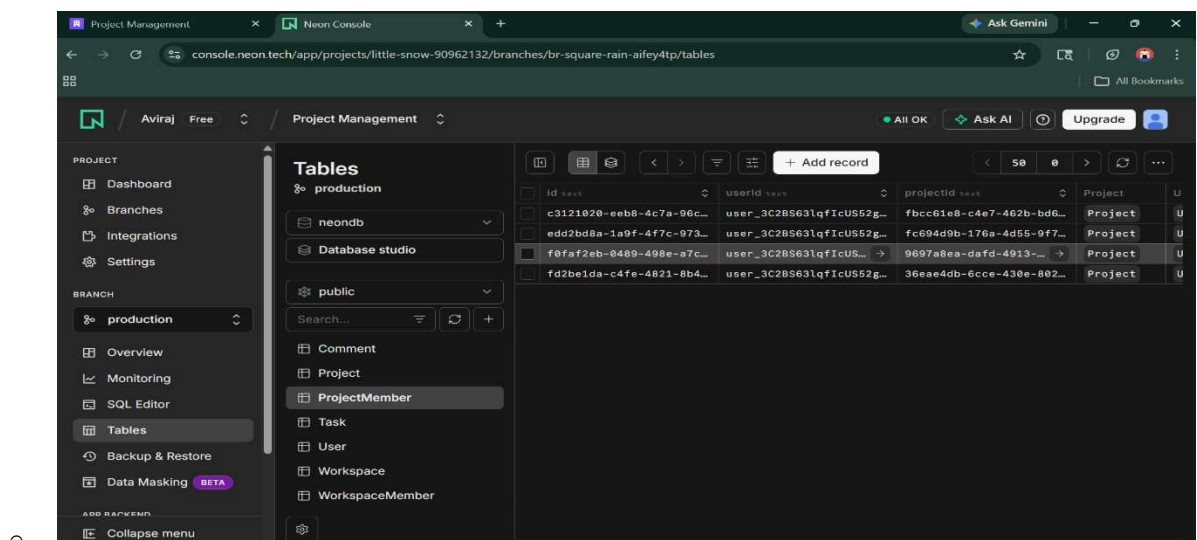
The screenshot shows the Neon Console interface with the 'production' branch selected. The 'Tables' section on the right lists several tables, with 'Workspace' highlighted. The 'Workspace' table has 2 records. The table structure is as follows:

id	name	slug	description
org_3CJ83To6WbcxpcGt7Z...	AvirajMGT	avirajmgt	NULL
org_3CJF4Rn73ev9ahGNC4...	Kamini Mam Medicaps	kamini-mam-medicaps	NULL

Fig 5.7 Workspace table

4. Project Members Table:

- *Columns:* id (PK), user id, project id, Project.
- *Description:* Information about members of the project.



The screenshot shows the Neon Console interface with the 'production' branch selected. The 'Tables' section on the right lists several tables, with 'ProjectMembers' highlighted. The 'ProjectMembers' table has 5 records. The table structure is as follows:

id	userId	projectId	Project
c3121820-eeb8-4c7a-96c...	user_3C2B8631qf1cUS52g...	fbcc61e8-c4e7-462b-bd6...	Project
edd2bd8a-1a9f-4f7c-973...	user_3C2B8631qf1cUS52g...	fc694d9b-176a-4d55-9f7...	Project
f0fa2eb-0489-498e-a7c...	user_3C2B8631qf1cUS...	9e97a8ea-dafd-4913-...	Project
fd2be1da-c4fe-4821-8b4...	user_3C2B8631qf1cUS52g...	36eae4db-6cce-430e-802...	Project

Fig 5.8 Project Members table

Chapter 6: Summary and Conclusions

The Project Management System is a comprehensive web-based solution designed to streamline project planning, task management, and team collaboration. By providing a centralized platform, the system effectively reduces manual effort and improves communication across various user roles, including Admins, Project Managers, and Team Members.

Key Conclusions:

- **Architectural Integrity:** The system utilizes a robust 3-tier architecture (Presentation, Application, and Data layers) to ensure a clear separation of concerns, which facilitates easier maintenance and upgrades.
- **Functional Breadth:** The system successfully addresses core project needs, including secure user authentication, project and task tracking, team role assignment, and document management.
- **Reliability and Security:** By implementing role-based access control, HTTPS encryption, and comprehensive testing strategies (Unit, Integration, and System testing), the platform ensures data integrity and a secure environment for organizational data.
- **Operational Efficiency:** The system is designed with high-priority performance requirements, aiming for response times within 2–3 seconds and the ability to handle large volumes of data without degradation.

Chapter 7: Future Scope

While the current SRS defines a solid foundation for project management, several areas have been identified for future enhancement to increase the system's competitive edge and utility:

- **Advanced Analytics:** Integration of a dedicated Reporting & Dashboard module to provide deeper visual insights into project health and team productivity.
- **Machine Learning Integration:** As suggested in the System Architecture diagram, future versions could implement Machine Learning models for resume analysis, job matching, or predictive task scheduling.
- **Real-time Communication:** Expanding the current comment-based feedback system into a real-time chat interface to further enhance immediate team collaboration.
- **Third-Party Integrations:** Connecting the system with external cloud services or productivity tools to create a more interconnected workflow.
- **Enhanced Mobility:** Further optimization for mobile devices and tablets beyond the current responsive web interface.

Appendix

Appendix A: Glossary

- **SRS:** Software Requirements Specification.
- **UI:** User Interface.
- **API:** Application Programming Interface.
- **PK/FK:** Primary Key and Foreign Key used in database design to ensure data uniqueness and referential integrity.

Appendix B: To Be Determined (TBD) List

- **Final Framework:** The specific selection of Node.js, Java, or PHP for the backend remains to be finalized.
- **Deployment:** The choice between a local server or specific cloud platforms (e.g., AWS, Azure) is pending.

Bibliography

1. Software Requirements Specification (SRS) for Project Management System, Prepared by Aviraj Singh and Avishka Vyas.

2. IEEE Standards for Software User Documentation (Reference for user manuals and setup guides).
3. Modern Web Development Documentation (HTML, CSS, JavaScript, and SQL/NoSQL Database standards).